

Universidad Europea de Madrid

Grado en Ingeniería de la Ciberseguridad

Técnicas de Hacking

Práctica 3

MITM y Suplantación

Detección de Envenenamiento ARP y DNS Snooping
mediante Sistemas de Detección de Intrusos Basados en Firmas

Asignatura: Técnicas de Hacking

Curso: 2025–2026

Entrega: 26 de mayo de 2026

Profesor: Alfredo Robledano Abasolo

Contacto:

alfredo.robledano@universidadeuropea.es

Resumen

Esta práctica implementa un sistema de monitorización capaz de detectar dos tipos de ataques de red habituales en entornos locales: el envenenamiento de tablas ARP y el reconocimiento mediante consultas DNS masivas a subdominios inexistentes (técnica asociada al ataque de Kaminsky).

En términos no técnicos, el envenenamiento ARP es como si alguien en una oficina cambiara todas las etiquetas de los buzones para que el correo llegue a la persona equivocada. El DNS Snooping es similar a alguien que llama repetidamente a una centralita telefónica preguntando por extensiones inventadas para mapear la estructura interna de la organización.

Para detectar ambos ataques se han desarrollado dos funciones en Python utilizando la librería Scapy [1], desplegadas sobre escenarios de red virtualizados con Docker Compose [2]. Los resultados demuestran que ambos detectores funcionan correctamente, lanzando alertas en tiempo real cuando se superan los umbrales definidos.

Palabras clave: ARP Spoofing, DNS Cache Poisoning, MITM, Kaminsky, IDS, Scapy, Docker.

Índice de Contenidos

1. Introducción	1
2. Marco Teórico	1
2.1. El protocolo ARP	1
2.1.1. Estructura del paquete ARP	1
2.1.2. ARP Spoofing y MITM	2
2.2. El protocolo DNS	2
2.2.1. Jerarquía DNS	2
2.2.2. Vulnerabilidades del protocolo DNS	2
2.2.3. El ataque de Kaminsky	3
2.3. Herramientas utilizadas	3
2.3.1. Scapy	3
2.3.2. bettercap	3
2.3.3. BIND9	3
2.3.4. Docker y Docker Compose	3
3. Metodología	4
3.1. Justificación del enfoque	4
3.2. Metodología de desarrollo	4
3.3. Entorno de laboratorio	4
3.4. Gestión de dependencias	4
4. Desarrollo	5
4.1. Parte 1: Detección de Envenenamiento ARP	5
4.1.1. Escenario de red	5
4.1.2. Estado legítimo de la tabla ARP	5
4.1.3. Ataque: ARP Spoofing con bettercap y Scapy	6
4.1.4. Función <code>alert_arpspoof</code>	6
4.1.5. Resultados: detección en tiempo real	7
4.2. Parte 2: Suplantación y Anomalías DNS	7
4.2.1. Escenario de red	7
4.2.2. Configuración DNS	7
4.2.3. Ataque: simulación del ataque de Kaminsky	8
4.2.4. Función <code>alert_dnssnooping</code>	8
4.2.5. Resultados: detección en tiempo real	9
5. Análisis del Tráfico Capturado	9
5.1. Análisis del tráfico ARP	9
5.1.1. Comparativa entre tráfico ARP legítimo y envenenado	9
5.1.2. Secuencia temporal del ataque ARP	9
5.2. Análisis del tráfico DNS	10
5.2.1. Estructura de un paquete DNS de consulta	10
5.2.2. Patrón de tráfico del ataque de Kaminsky	10
5.2.3. Secuencia temporal del ataque DNS	10
5.3. Limitaciones del entorno Docker	11
6. Resultados	11
6.1. Resumen de detecciones	11

6.2. Comparativa de detectores	11
7. Conclusiones	11
Bibliografía	12

Índice de Figuras

Figura 1 Contenedores del escenario ARP levantados correctamente	5
Figura 2 Tabla ARP legítima de la víctima antes del envenenamiento	6
Figura 3 Tabla ARP de la víctima tras el envenenamiento: la MAC del router ha sido sustituida por la del atacante	6
Figura 4 Alerta disparada por <code>alert_arpspoof</code> al detectar el cambio de MAC para <code>192.168.100.20</code>	7
Figura 5 Contenedores del escenario DNS levantados correctamente	7
Figura 6 Resolución correcta de <code>www.victima.local</code> a través del resolver	8
Figura 7 Alertas disparadas por <code>alert_dnssnooping</code> al superar el umbral de consultas DNS sospechosas	9

Índice de Tablas

Tabla 1 Campos del protocolo ARP relevantes para la detección de spoofing	2
Tabla 2 Componentes del sistema DNS y sus funciones	2
Tabla 3 Comparativa de enfoques de detección IDS	4
Tabla 4 Entorno de laboratorio utilizado	4
Tabla 5 Topología del escenario ARP	5
Tabla 6 Firmas de detección implementadas en <code>alert_arpspoof</code>	6
Tabla 7 Topología del escenario DNS	7
Tabla 8 Parámetros del generador de tráfico DNS malicioso	8
Tabla 9 Parámetros de detección de <code>alert_dnssnooping</code>	8
Tabla 10 Comparativa entre paquete ARP legítimo y envenenado	9
Tabla 11 Secuencia temporal del ataque ARP y detección	9
Tabla 12 Estructura de un paquete DNS query capturado con Scapy	10
Tabla 13 Comparativa entre tráfico DNS legítimo y ataque de Kaminsky	10
Tabla 14 Secuencia temporal del ataque DNS y detección	10
Tabla 15 Limitaciones del entorno Docker y soluciones adoptadas	11
Tabla 16 Resumen de resultados de detección	11
Tabla 17 Comparativa de los detectores implementados	11

1. Introducción

En un entorno hiperconectado, la integridad de las comunicaciones de red depende de protocolos diseñados décadas atrás, en una época en la que la seguridad no era una prioridad. Dos de los protocolos más fundamentales de las redes locales, ARP y DNS, presentan vulnerabilidades estructurales que permiten a un atacante situado en la misma red manipular el flujo de tráfico sin ser detectado.

El **envenenamiento ARP** (*ARP Spoofing*) explota la naturaleza *stateless* del protocolo ARP [3]: cualquier dispositivo acepta respuestas ARP aunque no haya realizado una petición previa, lo que permite a un atacante asociar su dirección MAC con la IP de otro dispositivo legítimo, posicionándose como intermediario en todas las comunicaciones (*Man-In-The-Middle*, MITM).

El **envenenamiento DNS** (*DNS Cache Poisoning*) aprovecha la misma naturaleza *stateless* del protocolo UDP sobre el que DNS opera [4]. El ataque de Kaminsky [5], descubierto en 2008, demostró que era posible envenenar la caché de un resolver DNS realizando consultas masivas a subdominios inexistentes e inyectando respuestas falsas en la sección de autoridad.

El objetivo de esta práctica es diseñar e implementar un sistema IDS (*Intrusion Detection System*) basado en firmas capaz de detectar ambos tipos de ataque en tiempo real, utilizando Python con Scapy [1] sobre escenarios de red virtualizados con Docker [2].

2. Marco Teórico

2.1. El protocolo ARP

El protocolo ARP (*Address Resolution Protocol*) fue definido en 1982 en el RFC 826 [3] y tiene como función resolver direcciones de capa de red (IP) a direcciones de capa de enlace (MAC) dentro de una misma red local. Su funcionamiento es sencillo: cuando un dispositivo A quiere comunicarse con un dispositivo B cuya IP conoce pero cuya MAC desconoce, emite un paquete de broadcast preguntando «¿quién tiene la IP X?». El dispositivo B responde con su MAC, y A almacena esta asociación en su caché ARP local para futuras comunicaciones.

El diseño original de ARP presenta dos características que lo hacen inherentemente vulnerable:

- **Stateless:** el protocolo no mantiene estado. Cualquier dispositivo puede enviar una respuesta ARP en cualquier momento, independientemente de si se realizó una petición previa.
- **Sin autenticación:** no existe ningún mecanismo que permita verificar que la respuesta ARP proviene del dispositivo legítimo.

Estas dos características, combinadas, permiten el ataque conocido como **ARP Spoofing** o **ARP Poisoning**, en el que un atacante envía respuestas ARP falsas para asociar su propia MAC con la IP de otro dispositivo.

2.1.1. Estructura del paquete ARP

Un paquete ARP contiene los siguientes campos relevantes para la detección:

Campo	Tamaño	Descripción
op	2 bytes	Operación: 1 = request, 2 = reply
hwsrc	6 bytes	MAC del emisor
psrc	4 bytes	IP del emisor
hwdst	6 bytes	MAC del destinatario
pdst	4 bytes	IP del destinatario

Tabla 1: Campos del protocolo ARP relevantes para la detección de spoofing

El campo `op=2` identifica un ARP reply. Un *Gratuitous ARP* se produce cuando `psrc == pdst`, es decir, cuando un dispositivo anuncia su propia IP sin que nadie lo haya solicitado.

2.1.2. ARP Spoofing y MITM

El ataque ARP Spoofing permite al atacante posicionarse como intermediario (*Man-In-The-Middle*) entre la víctima y el gateway. El flujo del ataque es el siguiente:

1. El atacante envía un ARP reply falso a la víctima, indicando que la IP del router corresponde a su propia MAC.
2. El atacante envía otro ARP reply falso al router, indicando que la IP de la víctima corresponde a su propia MAC.
3. Todo el tráfico entre víctima y router pasa ahora por el atacante, quien puede interceptarlo, modificarlo o simplemente reenviarlo.

Este ataque es la base de numerosas técnicas más avanzadas como el *SSL Stripping*, la inyección de contenido HTTP o el robo de credenciales.

2.2. El protocolo DNS

El Sistema de Nombres de Dominio (*Domain Name System*, DNS) fue definido en 1987 en los RFC 1034 y 1035 [4]. Su función es traducir nombres de dominio legibles por humanos (como `www.google.com`) a direcciones IP numéricas que los dispositivos utilizan para comunicarse.

2.2.1. Jerarquía DNS

DNS está organizado de forma jerárquica. Cuando un cliente realiza una consulta, el proceso de resolución implica varios actores:

Componente	Función
DNS Client	Aplicación que realiza la consulta (navegador, SO)
Resolver recursivo	Servidor que realiza la resolución completa en nombre del cliente
Root nameserver	Conoce los servidores TLD. Existen 13 clusters (<code>a-m.root-servers.net</code>)
TLD nameserver	Gestiona el TLD (<code>.com</code> , <code>.es</code> , <code>.org...</code>). Conoce los servidores autoritativos
Authoritative nameserver	Tiene la respuesta definitiva para el dominio consultado

Tabla 2: Componentes del sistema DNS y sus funciones

2.2.2. Vulnerabilidades del protocolo DNS

DNS opera sobre UDP en el puerto 53, heredando su naturaleza *stateless*. Un paquete DNS es aceptado como respuesta legítima si cumple tres condiciones:

1. Llega al puerto de origen correcto (aleatorio desde 2008).
2. Contiene el Transaction ID correcto (16 bits, aleatorio).
3. Tiene el flag QR=1 (respuesta).

Antes de 2008, el puerto de origen no era aleatorio, lo que reducía el espacio de búsqueda a solo 65.536 valores posibles (el Transaction ID). El ataque de Kaminsky [5] explotó esta debilidad de forma brillante.

2.2.3. El ataque de Kaminsky

Dan Kaminsky presentó en Black Hat 2008 un ataque que permitía envenenar la caché de cualquier resolver DNS sin necesidad de estar en la misma red. La técnica se basa en:

1. Realizar consultas masivas a subdominios inexistentes del dominio objetivo (p.ej., `abc123.victima.com`, `xyz789.victima.com...`).
2. Para cada consulta, enviar simultáneamente miles de respuestas falsas con diferentes Transaction IDs intentando adivinar el correcto.
3. En la sección de autoridad (*Authority Section*) de la respuesta falsa, incluir una entrada que envenena la caché del resolver para el dominio raíz.

La clave del ataque es que cada consulta a un subdominio inexistente obliga al resolver a realizar una nueva petición externa, reiniciando la «carrera» y permitiendo múltiples intentos sin esperar al TTL. Tras el descubrimiento, todos los resolvers modernos implementaron la aleatorización del puerto de origen, elevando el espacio de búsqueda a 2^{32} combinaciones.

2.3. Herramientas utilizadas

2.3.1. Scapy

Scapy [1] es una librería de Python que permite la creación, envío, captura y análisis de paquetes de red a bajo nivel. Es ampliamente utilizada en investigación de seguridad y permite construir paquetes arbitrarios en cualquier capa del modelo OSI. En esta práctica se utiliza tanto para el envío de paquetes ARP y DNS maliciosos como para la captura y análisis de tráfico en los detectores.

2.3.2. bettercap

bettercap [6] es una herramienta de seguridad ofensiva modular que implementa, entre otras funcionalidades, ataques ARP Spoofing con soporte para modo *full-duplex* (envenenamiento simultáneo de víctima y gateway). En esta práctica se utilizó para la fase de descubrimiento de hosts y como herramienta de ataque en el escenario ARP.

2.3.3. BIND9

BIND9 [7] (*Berkeley Internet Name Domain*) es el servidor DNS más utilizado en Internet. En esta práctica se empleó tanto como resolver recursivo como servidor autoritativo para la zona `victima.local`, proporcionando un entorno DNS realista sobre el que validar el detector.

2.3.4. Docker y Docker Compose

Docker [2] permite virtualizar entornos de red completos mediante contenedores ligeros. Docker Compose facilita la definición declarativa de escenarios multi-contenedor con topologías de red personalizadas. El uso de contenedores frente a máquinas virtuales reduce significativamente el tiempo de despliegue y el consumo de recursos, siendo ideal para entornos de laboratorio.

3. Metodología

3.1. Justificación del enfoque

La detección de ataques de red puede abordarse desde dos perspectivas complementarias:

Enfoque	Ventajas	Inconvenientes
Basado en firmas	Alta precisión, bajo coste computacional, fácil de auditar	No detecta variantes desconocidas del ataque
Basado en anomalías	Detecta ataques nuevos, adaptable	Mayor tasa de falsos positivos, más complejo

Tabla 3: Comparativa de enfoques de detección IDS

En esta práctica se ha optado por el enfoque basado en firmas, más apropiado para ataques bien conocidos y documentados como el ARP Spoofing y el DNS Snooping.

3.2. Metodología de desarrollo

El desarrollo de la práctica siguió la siguiente metodología:

1. **Despliegue del escenario:** definición de la topología en Docker Compose y verificación de la conectividad entre nodos.
2. **Establecimiento del estado legítimo:** captura del estado normal de la red (tabla ARP limpia, resolución DNS correcta) como línea base.
3. **Simulación del ataque:** ejecución del ataque con herramientas reales (bettercap, Scapy) para generar tráfico malicioso real.
4. **Implementación del detector:** desarrollo de las funciones de detección en Python con Scapy.
5. **Validación:** verificación de que el detector genera alertas correctas ante el tráfico malicioso.

3.3. Entorno de laboratorio

Componente	Versión / Detalle
Sistema operativo	Kali Linux (VirtualBox)
Docker	27.5.1
Docker Compose	2.32.4
Python	3.13
Scapy	2.7.0
bettercap	2.41.5
BIND9	9.18.39
uv	Gestor de dependencias Python

Tabla 4: Entorno de laboratorio utilizado

3.4. Gestión de dependencias

Las dependencias Python de cada parte se gestionaron con uv, inicializando proyectos independientes con `uv init --bare` en cada subdirectorio:

```
P3/  
├─ pyproject.toml  
└─ src/
```



```

├─ part1_arp/
│   ├── pyproject.toml ← scapy
│   └── uv.lock
└─ part2_dns/
    ├── pyproject.toml ← scapy
    └── uv.lock

```

4. Desarrollo

4.1. Parte 1: Detección de Envenenamiento ARP

4.1.1. Escenario de red

Se ha definido un escenario de red mediante Docker Compose [2] que incluye cuatro nodos distribuidos en dos redes virtuales:

Contenedor	Imagen	IP	Rol
victim	ubuntu:22.04	192.168.100.10	Nodo víctima
router	ubuntu:22.04	192.168.100.20 / .200.20	Gateway entre redes
webserver	nginx:alpine	192.168.200.30	Servidor web externo
attacker	kalilinux/kali-rolling	192.168.100.99	Nodo atacante

Tabla 5: Topología del escenario ARP

```

(kali@kali)-[~/Laboratorio-hacking-uem/P3/src/part1_arp]
$ docker compose up -d
[+] Running 6/6
✓ Network part1_arp_red_interna Created
✓ Network part1_arp_red_externa Created
✓ Container router Started
✓ Container webserver Started
✓ Container victim Started
✓ Container attacker Started

(kali@kali)-[~/Laboratorio-hacking-uem/P3/src/part1_arp]
$ docker compose ps
NAME                IMAGE              COMMAND                SERVICE    CREATED        STATUS        PORTS
attacker            kalilinux/kali-rolling "sleep infinity"      attacker   17 seconds ago Up 15 seconds
router              ubuntu:22.04       "sleep infinity"      router     17 seconds ago Up 15 seconds
victim              ubuntu:22.04       "sleep infinity"      victim     17 seconds ago Up 15 seconds
webserver           nginx:alpine        "/docker-entrypoint..." webserver  17 seconds ago Up 15 seconds  80/tcp

```

Figura 1: Contenedores del escenario ARP levantados correctamente

4.1.2. Estado legítimo de la tabla ARP

Antes de ejecutar el ataque, se registró el estado legítimo de la tabla ARP de la víctima:

```
(kali@kali)-[~/.../Laboratorio-hacking-uem/P3/src/part1_arp]
$ docker exec victim ping -c 3 192.168.100.20
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data.
64 bytes from 192.168.100.20: icmp_seq=1 ttl=64 time=2.96 ms
64 bytes from 192.168.100.20: icmp_seq=2 ttl=64 time=0.071 ms
64 bytes from 192.168.100.20: icmp_seq=3 ttl=64 time=0.055 ms

— 192.168.100.20 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2022ms
rtt min/avg/max/mdev = 0.055/1.029/2.962/1.366 ms

(kali@kali)-[~/.../Laboratorio-hacking-uem/P3/src/part1_arp]
$ docker exec victim arp -n
```

Address	HWtype	HWaddress	Flags	Mask	Iface
192.168.100.1	ether	02:42:34:86:37:58	C		eth0
192.168.100.20	ether	02:42:c0:a8:64:14	C		eth0

Figura 2: Tabla ARP legítima de la víctima antes del envenenamiento

Como se observa en Figura 2, la dirección IP 192.168.100.20 está correctamente asociada a la MAC 02:42:c0:a8:64:14, correspondiente al router legítimo.

4.1.3. Ataque: ARP Spoofing con bettercap y Scapy

El ataque se realizó en dos fases. Primero se utilizó bettercap [6] para el descubrimiento de hosts y la fase inicial del spoofing. Posteriormente, dado el aislamiento propio del bridge de Docker, se empleó Scapy [1] directamente desde el host para inyectar paquetes ARP falsos en el namespace de red de la víctima mediante nsenter:

```
Session Actions Edit View Help
Every 1.0s: docker exec victim arp -n
```

Address	HWtype	HWaddress	Flags	Mask	Iface
192.168.100.99	ether	02:42:c0:a8:64:63	C		eth0
192.168.100.20	ether	02:42:c0:a8:64:63	CM		eth0

Figura 3: Tabla ARP de la víctima tras el envenenamiento: la MAC del router ha sido sustituida por la del atacante

En Figura 3 se aprecia que la entrada para 192.168.100.20 muestra ahora la MAC 02:42:c0:a8:64:63 (atacante), con flag CM indicando una entrada estática modificada.

4.1.4. Función alert_arp spoof

Se implementó la función alert_arp spoof en Python con Scapy que monitoriza los paquetes ARP en tiempo real y detecta las siguientes anomalías:

Anomalía detectada	Descripción
Cambio de MAC	Una IP conocida aparece asociada a una MAC distinta a la registrada
Gratuitous ARP	ARP reply enviado sin solicitud previa o hacia broadcast

Tabla 6: Firmas de detección implementadas en alert_arp spoof

4.1.5. Resultados: detección en tiempo real

```
(kali@kali) [~/Laboratorio-hacking-uem/P3/src/part1_arp]
$ sudo nsenter -t $PID -n /home/kali/Desktop/Laboratorio-hacking-uem/P3/src/part1_arp/.venv/bin/python3 /home/kali/Desktop/Laboratorio-hacking-uem/P3/src/part1_arp/alert_arpspoof.py

IDS ARP - Detector de Envenenamiento de Caché ARP
Escuchando en la interfaz de red...

[15:46:45] [INFO] Entrada ARP registrada: 192.168.100.1 → 02:42:c0:a8:64:63
[15:46:58] [INFO] Entrada ARP registrada: 192.168.100.20 → 02:42:c0:a8:64:14
[15:47:12] [INFO] Entrada ARP registrada: 192.168.100.99 → 02:42:c0:a8:64:63
[15:47:17] [INFO] Entrada ARP registrada: 192.168.100.10 → 02:42:c0:a8:64:0a
[15:47:39] [ALERTA] ARP Spoofing detectado!
IP: 192.168.100.20
MAC real: 02:42:c0:a8:64:14
MAC falsa: 02:42:c0:a8:64:63
Posible atacante suplantando al dispositivo legítimo.
```

Figura 4: Alerta disparada por alert_arpspoof al detectar el cambio de MAC para 192.168.100.20

Como se muestra en Figura 4, el detector identifica correctamente el ataque, mostrando la IP afectada, la MAC legítima y la MAC falsa del atacante.

4.2. Parte 2: Suplantación y Anomalías DNS

4.2.1. Escenario de red

Se desplegó un segundo escenario Docker con tres nodos:

Contenedor	Imagen	IP	Rol
resolver	ubuntu/bind9	192.168.150.10	DNS Resolver recursivo
authns	ubuntu/bind9	192.168.150.20	Servidor autoritativo
client	ubuntu:22.04	192.168.150.30	Cliente DNS

Tabla 7: Topología del escenario DNS

```
(kali@kali) [~/Laboratorio-hacking-uem/P3/src/part2_dns]
$ cd ~/Desktop/Laboratorio-hacking-uem/P3/src/part2_dns
$ docker compose up -d
$ docker compose ps
[+] Running 5/5
  ✓ authns Pulled
  ✓ 196b5efbbc5f Pull complete
  ✓ 69b182ef14ea Pull complete
  ✓ 45f8831c1ae8 Pull complete
  ✓ resolver Pulled
[+] Running 4/4
  ✓ Network part2_dns_red_dns Created
  ✓ Container client Started
  ✓ Container authns Started
  ✓ Container resolver Started
NAME      IMAGE          COMMAND                  SERVICE    CREATED   STATUS    PORTS
authns    ubuntu/bind9:latest "docker-entrypoint.s..." authns     4 seconds ago Up 2 seconds 53/tcp, 953/tcp, 53/udp
client    ubuntu:22.04    "sleep infinity"         client     4 seconds ago Up 2 seconds 53/tcp, 953/tcp, 0.0.0.0:5353→53/udp, [::]:5353→53/udp
resolver  ubuntu/bind9:latest "docker-entrypoint.s..." resolver   4 seconds ago Up 1 second 53/tcp, 953/tcp, 0.0.0.0:5353→53/udp, [::]:5353→53/udp
```

Figura 5: Contenedores del escenario DNS levantados correctamente

4.2.2. Configuración DNS

Se configuró BIND9 [7] como resolver recursivo y servidor autoritativo para la zona victima.local. La correcta resolución del dominio se verificó mediante dig:

```

; <<>> DiG 9.18.39-0ubuntu0.22.04.3-Ubuntu <<>> @192.168.150.10 www.victima.local
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 53793
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 5aac5060e4f6d6ee010000006a0cc0b701f6dd7abd46e084 (good)
;; QUESTION SECTION:
;www.victima.local.                IN      A

;; ANSWER SECTION:
www.victima.local.        60      IN      A      192.168.150.20

;; Query time: 4 msec
;; SERVER: 192.168.150.10#53(192.168.150.10) (UDP)
;; WHEN: Tue May 19 19:57:43 UTC 2026
;; MSG SIZE rcvd: 90

(kali@kali)-[~/../Laboratorio-hacking-uem/P3/src/part2_dns]
$

```

Figura 6: Resolución correcta de `www.victima.local` a través del resolver

4.2.3. Ataque: simulación del ataque de Kaminsky

El script `generate_dns_traffic.py` genera consultas UDP con Scapy hacia el resolver usando subdominios aleatorios inexistentes:

Parámetro	Valor
Dominio base	<code>victima.local</code>
Subdominios	Aleatorios de 8 caracteres alfanuméricos
Transaction ID	Aleatorio (16 bits)
Puerto origen	Aleatorio (1024–65535)
Número de consultas	20
Intervalo	0.2 segundos

Tabla 8: Parámetros del generador de tráfico DNS malicioso

4.2.4. Función `alert_dnssnooping`

Parámetro	Valor
Ventana temporal	5 segundos
Umbral de consultas	5 consultas por IP
Filtro	UDP puerto 53, solo queries (QR=0)
Agrupación	Por IP origen

Tabla 9: Parámetros de detección de `alert_dnssnooping`

4.2.5. Resultados: detección en tiempo real

```
[kali@kali] ~/Laboratorio-hacking-um/P3/src/part2_dns
$ PID=$(docker inspect -f '{{.State.Pid}}' client)
sudo nsenter -t $PID -n /home/kali/Desktop/Laboratorio-hacking-um/P3/src/part2_dns/.venv/bin/python3 /home/kali/Desktop/Laboratorio-hacking-um/P3/src/part2_dns/alert_dnsnooping.pyPID=$(docker inspect -f '{{.State.Pid}}' client)
sudo nsenter -t $PID -n /home/kali/Desktop/Laboratorio-hacking-um/P3/src/part2_dns/.venv/bin/python3 /home/kali/Desktop/Laboratorio-hacking-um/P3/src/part2_dns/alert_dnsnooping.py
/home/kali/Desktop/Laboratorio-hacking-um/P3/src/part2_dns/.venv/bin/python3: can't open file '/home/kali/Desktop/Laboratorio-hacking-um/P3/src/part2_dns/alert_dnsnooping.pyPID=199419': [Errno 2] No such file or directory

IDS DNS - Detector de DNS Snooping / Kaminsky
Umbral: 5 consultas en 5s
Interfaz: todas

[16:05:52] [INFO] Consulta DNS: 192.168.150.30 + cz2aktcu.victima.local (1/5 en 5s)
[16:05:52] [INFO] Consulta DNS: 192.168.150.30 + w6m2zn1.victima.local (2/5 en 5s)
[16:05:52] [INFO] Consulta DNS: 192.168.150.30 + qnuh259.victima.local (3/5 en 5s)
[16:05:53] [INFO] Consulta DNS: 192.168.150.30 + 5lmidyoh.victima.local (4/5 en 5s)
[16:05:53] [INFO] Consulta DNS: 192.168.150.30 + c123wv5g.victima.local (5/5 en 5s)
[16:05:53] [ALERTA] DNS Snooping / Kaminsky detectado!
Origen: 192.168.150.30
Consultas en ventana: 5
Última consulta: c123wv5g.victima.local
Posible ataque de subdominios falsos o reconocimiento DNS.
[16:05:53] [INFO] Consulta DNS: 192.168.150.30 + 90pccdl0.victima.local (6/5 en 5s)
[16:05:53] [INFO] Consulta DNS: 192.168.150.30 + 0scalfi3.victima.local (7/5 en 5s)
[16:05:53] [INFO] Consulta DNS: 192.168.150.30 + 6kbp0zqa.victima.local (8/5 en 5s)
[16:05:54] [INFO] Consulta DNS: 192.168.150.30 + 742yirrf.victima.local (9/5 en 5s)
[16:05:54] [INFO] Consulta DNS: 192.168.150.30 + c3qfkyv9.victima.local (10/5 en 5s)
[16:05:54] [INFO] Consulta DNS: 192.168.150.30 + 2m8ja8fr.victima.local (11/5 en 5s)
[16:05:54] [INFO] Consulta DNS: 192.168.150.30 + 6q6u93t0.victima.local (12/5 en 5s)
[16:05:55] [INFO] Consulta DNS: 192.168.150.30 + dr3l98pq.victima.local (13/5 en 5s)
[16:05:55] [ALERTA] DNS Snooping / Kaminsky detectado!
Origen: 192.168.150.30
Consultas en ventana: 13
Última consulta: dr3l98pq.victima.local
Posible ataque de subdominios falsos o reconocimiento DNS.
[16:05:55] [INFO] Consulta DNS: 192.168.150.30 + 7izvo0dv.victima.local (14/5 en 5s)
[16:05:55] [INFO] Consulta DNS: 192.168.150.30 + hontdirb.victima.local (15/5 en 5s)
[16:05:55] [INFO] Consulta DNS: 192.168.150.30 + gV5pm2cy.victima.local (16/5 en 5s)
[16:05:56] [INFO] Consulta DNS: 192.168.150.30 + 32bqe0ly.victima.local (17/5 en 5s)
[16:05:56] [INFO] Consulta DNS: 192.168.150.30 + 8xcxc699.victima.local (18/5 en 5s)
[16:05:56] [INFO] Consulta DNS: 192.168.150.30 + 8c10pb0l.victima.local (19/5 en 5s)
[16:05:56] [INFO] Consulta DNS: 192.168.150.30 + 6k9t7aff.victima.local (20/5 en 5s)
```

Figura 7: Alertas disparadas por alert_dnsnooping al superar el umbral de consultas DNS sospechosas

5. Análisis del Tráfico Capturado

5.1. Análisis del tráfico ARP

5.1.1. Comparativa entre tráfico ARP legítimo y envenenado

Campo	Valor legítimo	Valor envenenado	Diferencia
op	2 (reply)	2 (reply)	—
psrc	192.168.100.20	192.168.100.20	Igual
hwsrc	02:42:c0:a8:64:14	02:42:c0:a8:64:63	MAC diferente ← firma
pdst	192.168.100.10	192.168.100.10	Igual

Tabla 10: Comparativa entre paquete ARP legítimo y envenenado

La firma de detección se basa en esta discrepancia: la IP de origen (psrc) es la del router legítimo, pero la MAC de origen (hwsrc) pertenece al atacante.

5.1.2. Secuencia temporal del ataque ARP

Tiempo	Origen	Evento
T+0s	Víctima	Ping al router → ARP request legítimo
T+1s	Router	ARP reply legítimo: MAC 02:42:c0:a8:64:14
T+2s	Detector	Registra: 192.168.100.20 → 02:42:c0:a8:64:14
T+60s	Atacante	ARP reply falso: MAC 02:42:c0:a8:64:63
T+60s	Detector	ALERTA: cambio de MAC detectado

Tabla 11: Secuencia temporal del ataque ARP y detección

5.2. Análisis del tráfico DNS

5.2.1. Estructura de un paquete DNS de consulta

Capa	Campo	Valor típico
IP	src	IP del cliente
IP	dst	IP del resolver (192.168.150.10)
UDP	sport	Puerto aleatorio > 1024
UDP	dport	53
DNS	id	Transaction ID aleatorio (0–65535)
DNS	qr	0 (query)
DNS	rd	1 (recursión deseada)
DNS	qname	Nombre consultado
DNS	qtype	A (dirección IPv4)

Tabla 12: Estructura de un paquete DNS query capturado con Scapy

5.2.2. Patrón de tráfico del ataque de Kaminsky

Característica	Tráfico legítimo	Tráfico malicioso (Kaminsky)
Frecuencia	Baja, esporádica	Alta, ráfagas sostenidas
Subdominios	Conocidos y repetidos	Aleatorios, nunca repetidos
Respuestas	NOERROR	NXDOMAIN
Transaction ID	Secuencial o bajo	Completamente aleatorio
Puerto origen	Variable	Aleatorio en cada paquete

Tabla 13: Comparativa entre tráfico DNS legítimo y ataque de Kaminsky

5.2.3. Secuencia temporal del ataque DNS

Tiempo	Evento	Detalle
T+0.0s	Consulta 1/20	cz24ktcu.victima.local (TID=40105)
T+0.2s	Consulta 2/20	q2isv8uw.victima.local (TID=62661)
T+0.4s	Consulta 3/20	61zy6vwl.victima.local (TID=34354)
T+0.6s	Consulta 4/20	mhim18yu.victima.local (TID=18855)
T+0.8s	Consulta 5/20	5zf1pmlc.victima.local (TID=46796)
T+0.8s	ALERTA	Umbral superado: 5 consultas en 5s
T+1.0s–4.0s	Consultas 6–20	Continúan las consultas a subdominios aleatorios
T+2.6s	ALERTA	Segunda alerta: 13 consultas en ventana

Tabla 14: Secuencia temporal del ataque DNS y detección

5.3. Limitaciones del entorno Docker

Limitación	Causa	Solución adoptada
ARP filtering en bridges	Docker filtra paquetes ARP entre contenedores	Uso de <code>nsenter</code> en namespace de red de la víctima
Scapy no ve tráfico Docker	El bridge Docker aísla el tráfico del host	Ejecución de scripts dentro del namespace
pip en contenedores Kali	Entorno Python externamente gestionado	Flag <code>--break-system-packages</code>

Tabla 15: Limitaciones del entorno Docker y soluciones adoptadas

6. Resultados

6.1. Resumen de detecciones

Parte	Ataque simulado	Alertas generadas	Resultado
1 — ARP	ARP Spoofing (bettercap + Scapy)	1 alerta de cambio de MAC	✓ Detectado
2 — DNS	DNS Snooping / Kaminsky (20 consultas)	2 alertas por umbral	✓ Detectado

Tabla 16: Resumen de resultados de detección

6.2. Comparativa de detectores

Detector	Protocolo	Capa OSI	Método	Falsos positivos
<code>alert_arp spoof</code>	ARP	Capa 2	Firma: cambio MAC	Bajos
<code>alert_dnssnooping</code>	DNS	Capa 7	Umbral de volumen	Medios

Tabla 17: Comparativa de los detectores implementados

7. Conclusiones

Esta práctica ha permitido comprender en profundidad dos de los vectores de ataque más clásicos y persistentes en redes locales: el envenenamiento ARP y el DNS Snooping/Kaminsky. Se ha demostrado que es posible detectar ambos ataques en tiempo real mediante análisis de tráfico basado en firmas, implementado con herramientas de código abierto como Scapy [1]. Los detectores desarrollados presentan características complementarias:

- `alert_arp spoof` detecta cambios inesperados en la asociación IP→MAC con baja tasa de falsos positivos, al basarse en una firma determinista y bien definida.
- `alert_dnssnooping` detecta ráfagas de consultas DNS mediante umbral de volumen, técnica propia de los IDS de red modernos, con mayor sensibilidad pero también mayor riesgo de falsos positivos en entornos con alto volumen de consultas legítimas.

Como limitaciones identificadas, el entorno Docker introduce restricciones en la propagación de paquetes ARP entre contenedores, requiriendo el uso de `nsenter` para acceder al namespace de red correcto. En una red física real, ambos detectores operarían sin estas restricciones.

Como trabajo futuro se propone la integración de DNSSEC [4] para mitigar el envenenamiento DNS mediante firma digital de registros, y el uso de entradas ARP estáticas o *Dynamic ARP*

Inspection (DAI) en switches gestionados para prevenir el ARP Spoofing a nivel de infraestructura.

Bibliografía

- [1] P. Biondi, «Scapy: Packet crafting for Python2 and Python3». [En línea]. Disponible en: <https://scapy.net/>
- [2] Docker Inc., «Docker: Accelerated Container Application Development». [En línea]. Disponible en: <https://www.docker.com/>
- [3] D. C. Plummer, «RFC 826: An Ethernet Address Resolution Protocol». [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc826>
- [4] P. Mockapetris, «RFC 1034: Domain Names - Concepts and Facilities». [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc1034>
- [5] D. Kaminsky, «It's the End of the Cache As We Know It», en *Black Hat USA*, 2008. [En línea]. Disponible en: <https://www.blackhat.com/presentations/bh-jp-08/bh-jp-08-Kaminsky/BlackHat-Japan-08-Kaminsky-DNS08-Japan.pdf>
- [6] S. Mariani, «bettercap: The Swiss Army knife for 802.11, BLE, IPv4 and IPv6 networks reconnaissance and MITM attacks». [En línea]. Disponible en: <https://www.bettercap.org/>
- [7] ISC, «BIND 9: Versatile, classic, complete name server software». [En línea]. Disponible en: <https://www.isc.org/bind>